

Mission « Pointeuse 2.0 » — COGIP

Q1

Trois inconvénients d'une ancienne pointeuse électromécanique :

- Maintenance difficile et coûteuse.
- Données peu faciles à exploiter.
- Évolution et connexion à d'autres systèmes limitées.

Q2

Le salarié utilise **Badger pour obtenir l'autorisation d'accès**.

Le système inclut l'**identification du salarié**, la **vérification des droits**, l'**enregistrement du pointage** et l'**affichage du résultat**.

Le responsable RH peut **consulter et modifier les droits d'accès**.

Q3

Le programme permet de faire clignoter la LED intégrée de l'ESP32.

L'identifiant affiché dans le moniteur série est : **[à compléter avec ton Chip ID]**.

À 115200 bauds, le moniteur série affiche correctement les caractères. À une autre vitesse, les caractères deviennent illisibles car la vitesse de communication est différente.

Q4

- `digitalWrite(2, HIGH)` : allume la LED sur GPIO 2.
- `digitalWrite(2, LOW)` : éteint la LED.
- `pinMode(2, OUTPUT)` : configure GPIO 2 en sortie.
- `uint64_t chipid` : crée une variable entière non signée sur 64 bits pour stocker l'identifiant de l'ESP32.

Q5

- **SCK** : horloge générée par le maître.
- **MOSI** : données envoyées du maître vers l'esclave.
- **MISO** : données envoyées de l'esclave vers le maître.
- **SS/NSS** : sélection de l'esclave.

Q6

Le SPI fonctionne en **full-duplex**, car les données peuvent circuler simultanément dans les deux directions grâce à MOSI et MISO.

Pour deux esclaves, le maître utilise les lignes communes **SCK, MOSI et MISO**, ainsi qu'une ligne **SS différente pour chaque esclave**, soit **5 connexions** au total.

Q7

Broche RC522	Direction	Rôle
NSS	Entrée	Sélection du lecteur
SCK	Entrée	Horloge SPI
MOSI	Entrée	Données venant de l'ESP32
MISO	Sortie	Données envoyées vers l'ESP32
IRQ	Sortie	Signal d'interruption
GND	Alimentation	Masse
RST	Entrée	Reset du RC522
3.3V	Alimentation	Alimentation 3,3 V

Q8

L'antenne est la **boucle de cuivre** située sur le module RC522. Elle permet de communiquer sans contact avec le badge RFID et de récupérer son identifiant.

La portée typique est de **quelques centimètres**, selon le badge et les conditions.

Q9

RC522	ESP32
NSS	GPIO 5
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19
IRQ	Non connecté
GND	GND
RST	GPIO 4
3.3V	3.3V

Q10

Le câblage ESP32 ↔ RC522 a été réalisé conformément au schéma et vérifié avant la mise sous tension.

Q11

Le programme ReadNUID permet de détecter un badge et d'afficher son NUID dans le moniteur série.

Résultat obtenu : [coller ici le résultat du moniteur série].

Q12

Les NUID relevés sont :

- Badge A : [871B8E62]
- Badge B : [F90CE6A3]

Le nombre d'octets dépend du badge utilisé. Le « N » de NUID signifie **Non-Unique**, car l'identifiant n'est pas considéré comme un moyen d'authentification suffisamment sécurisé à lui seul.

Q13

La LED intégrée est allumée pendant 500 ms après une lecture réussie grâce à :

```
digitalWrite(2, HIGH);
```

puis après 500 ms :

```
digitalWrite(2, LOW);
```

Q14

IRQ signifie *Interrupt Request*.

- Le **polling** consiste à vérifier régulièrement l'état du lecteur.
- Une **interruption** permet au lecteur de signaler directement un événement au microcontrôleur.

Q15

L'utilisation de l'IRQ permet :

- de réduire la consommation d'énergie en évitant une scrutation permanente ;
- de laisser l'ESP32 effectuer d'autres tâches pendant l'attente d'un badge.

Q16

On peut configurer une interruption sur l'ESP32 avec `attachInterrupt()` et un front descendant :

```
pinMode(IRQ_PIN, INPUT_PULLUP);  
attachInterrupt(digitalPinToInterrupt(IRQ_PIN), badgeInterrupt, FALLING);
```

La fonction `badgeInterrupt()` est appelée lorsqu'un signal descendant apparaît sur la broche IRQ.

Q17

Le volume `cogip-data` permet de conserver les données PostgreSQL même si le conteneur est supprimé.

Sans volume, les données seraient perdues avec la suppression du conteneur.

Q18

Mettre un mot de passe directement dans un fichier est dangereux car toute personne ayant accès au fichier peut le récupérer.

Une meilleure solution est d'utiliser une **variable d'environnement** ou un fichier de configuration sécurisé.

Q19

Le modèle peut être organisé ainsi :

- **users** : contient les salariés.
- **badges** : chaque badge appartient à un utilisateur.
- **zones** : contient les zones protégées.
- **access_rights** : associe un utilisateur à une zone.
- **access_logs** : enregistre chaque passage.

Un utilisateur peut avoir plusieurs badges et plusieurs droits. Une zone peut être accessible à plusieurs utilisateurs.

Le NUID n'est pas une bonne clé primaire car il peut être copié ou modifié. Il vaut mieux utiliser un identifiant interne pour le badge.

Q20

`requirements.txt` contient les dépendances Python nécessaires au projet. Il permet d'installer automatiquement les bibliothèques nécessaires avec :

```
pip install -r requirements.txt
```

Q21

`UniqueConstraint("user_id", "zone_id")` empêche de créer deux fois le même droit pour un utilisateur dans une même zone.

`unique=True` sur `nuid` empêche deux badges d'avoir le même NUID.

`index=True` améliore les recherches sur le NUID.

Q22

`--host 0.0.0.0` permet au serveur d'être accessible depuis les autres appareils du réseau.

`127.0.0.1` limite l'accès à l'ordinateur lui-même.

Le risque est d'exposer le serveur au réseau. Il faut donc limiter les accès avec un pare-feu, une authentification et éventuellement HTTPS.

Q23

Dans notre réalisation, les utilisateurs, badges et droits sont configurés dans l'application.

Exemple :

- Utilisateur 1 → Badge A → Salle serveur → **autorisé**
- Utilisateur 2 → Badge B → Salle serveur → **non autorisé**

Q24

La commande fournie permet de vérifier les badges présents dans PostgreSQL :

```
docker exec -it cogip-db psql -U cogip -d pointeuse -c "SELECT *  
FROM badges;"
```

Une requête permettant d'afficher les informations demandées est :

```
SELECT u.nom, u.prenom, b.nuid, z.nom  
FROM users u  
JOIN badges b ON b.user_id = u.id  
JOIN access_rights ar ON ar.user_id = u.id  
JOIN zones z ON z.id = ar.zone_id;
```

Q25

Le programme ESP32 a été modifié pour :

- se connecter au Wi-Fi ;
- détecter un badge RFID ;
- récupérer son NUID ;
- envoyer le NUID au serveur FastAPI avec une requête HTTP.

Q26

Sous Windows, l'adresse IP du serveur peut être obtenue avec :

```
ipconfig
```

L'adresse IP fixe ou une réservation DHCP est importante car l'ESP32 doit toujours connaître l'adresse du serveur pour envoyer les requêtes.

Dans notre projet, l'ESP32 utilise l'adresse du serveur : **192.168.1.160**.

Q27

N°	Badge	Zone	Résultat attendu
1	Badge A autorisé	Salle serveur	Autorisé
2	Badge inconnu	Salle serveur	Refusé / Inconnu
3	Badge B autorisé	Salle serveur	Autorisé
4	Badge B désactivé	Salle serveur	Refusé

N°	Badge	Zone	Résultat attendu
5	Badge A autorisé	Salle serveur	Autorisé

Les résultats obtenus doivent être complétés après les essais réels.

Q28

Avec HTTP, les données envoyées entre l'ESP32 et le serveur ne sont pas chiffrées.

Une personne présente sur le réseau pourrait donc observer les informations échangées.

La solution serait d'utiliser **HTTPS/TLS** afin de chiffrer les communications.

Q29

La page web interroge `/api/last` toutes les secondes : c'est du **polling**.

Cela génère des requêtes répétées, même lorsqu'aucun nouveau pointage n'a eu lieu.

Une solution serait d'utiliser **WebSocket** ou une technologie de type **Server-Sent Events (SSE)** pour envoyer directement les nouvelles informations au navigateur.

Q30

Si le serveur est indisponible, l'ESP32 ne peut pas vérifier le badge et le pointage n'est pas enregistré.

Pour améliorer la continuité de service, on peut ajouter une mémoire locale sur l'ESP32 pour stocker temporairement les pointages puis les envoyer au serveur lorsque la connexion revient.

Q31

Lors de l'anticollision ISO 14443A, certaines informations permettant d'identifier le badge sont échangées sans chiffrement.

Le NUID ne doit donc pas être considéré comme un secret. Utiliser uniquement le NUID comme authentification n'est pas suffisamment sécurisé.

Q32

Le NUID d'un badge n'est pas secret et peut être lu à proximité.

Les **magic cards** permettent de modifier certains UID. Elles peuvent donc faciliter le clonage d'un badge basé uniquement sur son UID.

Q33

Une attaque par **rejeu** consiste à enregistrer une communication valide puis à la reproduire plus tard.

Une attaque par **relais** consiste à transmettre en temps réel les communications entre un badge et un lecteur éloigné afin de tromper le système.

Q34

Le prototype est adapté à un usage peu critique, comme le pointage des salariés ou l'identification simple.

Il ne doit pas être utilisé seul pour protéger une zone sensible comme une salle des coffres.

Le NUID peut être copié et n'est pas un secret.

Une personne malveillante pourrait donc imiter un badge.

Pour une zone sensible, il faudrait utiliser une authentification RFID plus sécurisée.

Il faudrait également chiffrer les communications avec le serveur.

Le prototype constitue donc une bonne solution pédagogique et de pointage, mais pas une solution de haute sécurité.

Q35

Exigence	Vérification
Lecture RFID	✓ Réalisée avec le RC522
Connexion réseau	✓ Wi-Fi ESP32
Identification du salarié	✓ Serveur FastAPI
Contrôle des droits	✓ Gestion des badges/droits
Enregistrement des passages	✓ Base de données
Affichage web	✓ Page web
Gestion des droits RH	✓ API/application
Sécurité forte	⚠ Non totalement couverte
Chiffrement des communications	⚠ HTTP utilisé, HTTPS à ajouter

La solution répond donc à la majorité des exigences fonctionnelles, mais certaines exigences de sécurité doivent encore être améliorées.